



ROOT CAUSE ANALYSIS

VIRTUAL CONTROL
VMWARE CLOUD FOUNDATION SOLUTIONS

VCF Offline Depot RCA Report

Complete root cause analysis and recovery walkthrough for VCF offline depot configuration failures, with every command, output, and verified fix.

OFFLINE DEPOT

RCA

BUNDLE MANAGEMENT

RECOVERY

VCF 9.0

VMware Cloud Foundation

Root Cause Analysis — VCF Offline Depot Configuration

PowerShell users: Every bash block in this document that targets a REST API or other client-runnable command has a paste-ready PowerShell sibling block underneath it. For the full substitution table — `curl` → `Invoke-RestMethod`, `base64` auth headers, `here-docs` → `here-strings`, `jq` / `python3 -m json.tool` → `ConvertFrom-Json` / `ConvertTo-Json -Depth N`, etc. — see [POWERSHELL-TRANSLATION-REFERENCE.md](#). Blocks targeting the appliance shell only (`systemctl`, `keytool`, `openssl s_client` inside the VM, etc.) are flagged "Run on appliance only" with no PS sibling — they cannot run from a Windows workstation.

Table of Contents

1. Executive Summary
2. Environment Reference
3. Problem Statement
4. Phase 1 — Start the HTTPS Depot Server
5. Phase 2 — First Failure: HTTP 401
6. Phase 3 — Remove Authentication and Retest
7. Phase 4 — Second Failure: Dialog Closes, No Error
8. Phase 5 — Attempt SSH to VCF Operations
9. Phase 6 — Import Certificate via vCenter Guest Operations API
10. Phase 7 — Verify Certificate Import from Inside the VM
11. Phase 8 — Third Failure: Depot URL Invalid
12. Phase 9 — Analyze Request Logs
13. Phase 10 — Correct the Serving Directory and URL
14. Phase 11 — Final Validation
15. Summary of All Four Root Causes
16. Lessons Learned
17. Appendix A — Final HTTPS Server Script
18. Appendix B — Certificate Import Script
19. Appendix C — Diagnostic Verification Script
20. Document Information

1. Executive Summary

Where commands run in this RCA. Three environments dominate:

- **Windows workstation (PowerShell or CMD)** — running the Python HTTPS server, OpenSSL cert generation, file system operations on the depot directory. Most commands prefixed `ps c:\>` or run via PowerShell.
- **VCF Installer (SSH)** — for testing depot connectivity, certificate import, FIPS workarounds. SSH as root.

- **VCF Operations / Fleet Management UI (browser)** — depot configuration form, sync triggers, status checks at `https://<vcf-ops-fqdn>/` .

Each section header notes the environment in parentheses (e.g., "Diagnostic Commands (VCF Installer)"). Replace any credential placeholder like `<your-password>` with values from your vault.

On March 23, 2026, the VCF Operations Fleet Management offline depot was configured to use an HTTPS web server hosted on a Windows workstation. The initial configuration failed silently. Through systematic troubleshooting, four root causes were identified and resolved. This document records every command executed, every output received, every error encountered, and every fix applied — in the exact order they occurred — so that the process can be duplicated.

2. Environment Reference

VCF Components:

Role	IP Address	FQDN
Windows Workstation (Depot Server)	192.168.1.160	—
VCF Operations (vcf-ops VM)	192.168.1.77	—
Fleet Manager	192.168.1.78	—
Remote Collector	192.168.1.79	—
vCenter Server	192.168.1.69	vcenter.lab.local
SDDC Manager	192.168.1.68	sddc-manager.lab.local
NSX VIP	192.168.1.70	nsx-vip.lab.local

Depot Server Details:

Property	Value
OS	Windows 11 Pro
Python	3.14
Script	E:\VCF-Depot\https_server.py
Port	TCP 8443
TLS	1.2 through 1.3
Certificate	E:\VCF-Depot\server.crt
Private Key	E:\VCF-Depot\server.key
Cert CN/SAN	192.168.1.160
Cert Expiry	Feb 5, 2027
Cert SHA-256	AA:A7:BB:F8:09:0C:D5:CF:18:30:33:D5:8E:5F:CF:60:D6:F4:A7:55:2E:98:05:A9:D4:94:1B:4C:AD:CA:2F:18

Required Directory Structure on the Depot Server:

Fleet Management validates the depot by requesting this file at the configured URL:

```
<depot-url>/PROD/metadata/manifest/v1/vcfManifest.json
```

This file comes from the `vcf-*-offline-depot-metadata.zip` downloaded from Broadcom Support Portal. The directory tree must look like this:

```
E:\VCF-Depot\  
  PROD\  
    metadata\  
      manifest\  
        v1\  
          vcfManifest.json      <-- This file MUST exist  
          productVersionCatalog\  
            v1\  
              productVersionCatalog.json  
          vsan\  
            hcl\  
          COMP\  
          server.crt  
          server.key  
          https_server.py
```

3. Problem Statement

Goal: Configure the VCF Operations Fleet Management offline depot to point to a local HTTPS web server so management packs can be installed and upgraded without internet access.

Location in UI: VCF Operations > Fleet Management > Depot Configuration > Offline Depot > CONFIGURE

What happened: After filling in the "Configure Offline Depot" dialog (OfflineDepotType = Webserver, Repository URL, Username, Password) and clicking OK, the dialog closed but the settings did not save. No error message was displayed.

4. Phase 1 — Start the HTTPS Depot Server

4.1 What We Did

Opened a terminal on the Windows workstation (192.168.1.160) and started the HTTPS depot server.

Command:

```
# bash on workstation (git-bash / WSL)  
cd /e/VCF-Depot && py https_server.py
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation  
Set-Location 'E:\VCF-Depot'  
py .\https_server.py
```

Output:

```
VCF Offline Depot Server
=====
Serving: E:\VCF-Depot
URL: https://192.168.1.160:8443/
Credentials: admin / admin
TLS: 1.2 - 1.3
Press Ctrl+C to stop
```

4.2 Verify the Server Is Listening

Command:

```
# bash on workstation (git-bash / WSL)
netstat -an | grep 8443
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Get-NetTCPConnection -LocalPort 8443 -ErrorAction SilentlyContinue |
    Select-Object LocalAddress, LocalPort, State, OwningProcess
```

Output:

```
TCP    0.0.0.0:8443    0.0.0.0:0      LISTENING
```

This confirms the server started and is listening on port 8443 on all interfaces.

4.3 Test the Server from the Workstation

Command:

```
# bash on workstation
curl -sk https://192.168.1.160:8443/
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true } # PS 5.1 self-signed
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
Invoke-RestMethod -Uri 'https://192.168.1.160:8443/'
# OR: curl.exe -sk https://192.168.1.160:8443/
```

Output:

```
<!DOCTYPE HTML>
<html lang="en">
<head><title>Directory listing for /</title></head>
<body>
<h1>Directory listing for /</h1>
<ul>
<li><a href="PROD/">PROD</a></li>
<li><a href="server.crt">server.crt</a></li>
<li><a href="server.key">server.key</a></li>
...
```

```
</ul>
</body>
</html>
```

The server responded with a directory listing. At this point, the server was working correctly from the workstation.

Note: The depot files were later reported to be on the `F:\` drive. The server was changed to serve `F:\` instead. This introduced Root Cause 3 (covered in Phase 8).

5. Phase 2 — First Failure: HTTP 401 Authentication Required

5.1 What the User Did

1. Opened VCF Operations in a browser
2. Navigated to: **Fleet Management > Depot Configuration**
3. Under "Offline Depot", clicked **CONFIGURE**
4. Selected **OfflineDepotType: Webserver**
5. Entered **Repository URL:** `https://192.168.1.160:8443/`
6. Entered **Username:** `admin`
7. Entered **Password:** `admin`
8. Clicked **OK**

5.2 What Happened

The user reported: "**Authentication required, status code : 401**"

5.3 Why This Happened

The HTTPS server script (`https_server.py`) had Basic Authentication enabled. Every incoming request was checked by an `authenticate()` method. If the `Authorization` header was missing or contained wrong credentials, the server returned HTTP 401.

The original code that caused this (lines 26-34 of `https_server.py`):

```
def do_HEAD(self):
    if not self.authenticate():
        return
    super().do_HEAD()

def do_GET(self):
    if not self.authenticate():
        return
    super().do_GET()

def do_POST(self):
    if not self.authenticate():
        return
    # ... handle POST ...
```

The `authenticate()` method (lines 47-62):

```
def authenticate(self):
    auth_header = self.headers.get('Authorization')
```

```

    if auth_header is None:
        self.send_auth_request() # Returns HTTP 401
        return False
    try:
        auth_type, credentials = auth_header.split(' ', 1)
        if auth_type.lower() != 'basic':
            self.send_auth_request()
            return False
        decoded = base64.b64decode(credentials).decode('utf-8')
        username, password = decoded.split(':', 1)
        if username == USERNAME and password == PASSWORD:
            return True
    except Exception:
        pass
    self.send_auth_request()
    return False

```

The `send_auth_request()` method (lines 64-71):

```

def send_auth_request(self):
    self.send_response(401)
    self.send_header('WWW-Authenticate',
                    'Basic realm="VCF Depot"')
    self.send_header('Content-type', 'text/html')
    self.send_header('Content-Length', '23')
    self.send_header('Connection', 'close')
    self.end_headers()
    self.wfile.write(b'Authentication required')

```

Fleet Management was not sending the credentials in the format the server expected, so every request was rejected with HTTP 401.

6. Phase 3 — Remove Authentication and Retest

6.1 The Fix

Edited `E:\VCF-Depot\https_server.py` and removed the `authenticate()` call from all three HTTP methods.

Before (authentication enforced):

```

def do_HEAD(self):
    if not self.authenticate():
        return
    super().do_HEAD()

def do_GET(self):
    if not self.authenticate():
        return
    super().do_GET()

def do_POST(self):
    if not self.authenticate():
        return
    content_length = int(
        self.headers.get('Content-Length', 0))

```

After (authentication removed):

```

def do_HEAD(self):
    super().do_HEAD()

def do_GET(self):
    super().do_GET()

def do_POST(self):
    content_length = int(
        self.headers.get('Content-Length', 0))

```

6.2 Kill the Old Server and Restart

Step 1 — Find and kill the process using port 8443:

```

# bash on workstation (git-bash)
taskkill /F /PID $(netstat -ano | \
    grep ':8443.*LISTENING' | awk '{print $5}')

```

PowerShell equivalent:

```

# PowerShell 5.1+ on Windows workstation
$pid8443 = (Get-NetTCPConnection -LocalPort 8443 -State Listen -ErrorAction
    SilentlyContinue).OwningProcess
if ($pid8443) { Stop-Process -Id $pid8443 -Force }

```

Output:

```

SUCCESS: The process with PID 2548 has been terminated.

```

Step 2 — Verify port is free:

```

# bash on workstation
netstat -an | grep 8443

```

PowerShell equivalent:

```

# PowerShell 5.1+ on Windows workstation
Get-NetTCPConnection -LocalPort 8443 -ErrorAction SilentlyContinue |
    Select-Object LocalAddress, LocalPort, State, OwningProcess

```

Output: (no output — port is free)

Step 3 — Restart the server:

```

# bash on workstation
cd /e/VCF-Depot && py https_server.py

```

PowerShell equivalent:

```

# PowerShell 5.1+ on Windows workstation
Set-Location 'E:\VCF-Depot'
py .\https_server.py

```

Step 4 — Verify it is listening:


```
# bash on workstation
netstat -an | grep 8443
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Get-NetTCPConnection -LocalPort 8443 -ErrorAction SilentlyContinue |
    Select-Object LocalAddress, LocalPort, State, OwningProcess
```

Output:

```
TCP    0.0.0.0:8443    0.0.0.0:0      LISTENING
```

6.3 Verify No Authentication Required

Command:

```
# bash on workstation
curl -sk -o /dev/null -w "%{http_code}" \
https://192.168.1.160:8443/
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true } # PS 5.1 self-signed
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
(Invoke-WebRequest -Uri 'https://192.168.1.160:8443/' -UseBasicParsing).StatusCode
# OR: curl.exe -sk -o NUL -w "%{http_code}" https://192.168.1.160:8443/
```

Output:

```
200
```

The server now returns HTTP 200 without requiring any credentials. Root Cause 1 is resolved.

7. Phase 4 — Second Failure: Dialog Closes, No Error

7.1 What the User Did

1. Went back to VCF Operations > Fleet Management > Depot Configuration
2. Clicked CONFIGURE under Offline Depot
3. Selected **Webserver**
4. Entered the URL and credentials
5. Checked "I accept the imported certificate"
6. Clicked OK

7.2 What Happened

The dialog closed. The settings did not save. No error message was displayed. The depot still showed "Offline depot connection not set up."

7.3 Initial Theory

The most likely cause of a silent failure after removing authentication is a **TLS certificate trust issue**. The depot server uses a self-signed certificate. VCF Operations (a Java-based application) validates TLS certificates against its Java trust store. If the certificate is not trusted, the SSL handshake fails and the connection is rejected — but the UI does not show an error.

8. Phase 5 — Attempt SSH to VCF Operations Appliance

To import the depot certificate into the VCF Operations trust store, we first tried to SSH into the appliance directly.

8.1 First SSH Attempt

Command:

```
# bash on workstation (git-bash)
ssh -o ConnectTimeout=5 \
    -o StrictHostKeyChecking=no \
    -o BatchMode=yes \
    root@192.168.1.77 "echo OK"
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation (Win10/11 ships OpenSSH client)
ssh -o ConnectTimeout=5 `
    -o StrictHostKeyChecking=no `
    -o BatchMode=yes `
    root@192.168.1.77 "echo OK"
```

Output (FAILED):

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION
@    HAS CHANGED!
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Offending ECDSA key in
  C:\Users\valca\.ssh\known_hosts:8

Password authentication is disabled to
  avoid man-in-the-middle attacks.

root@192.168.1.77: Permission denied
  (publickey,password,keyboard-interactive).
```

Two problems identified:

1. The SSH host key for 192.168.1.77 in `known_hosts` was stale (the appliance was redeployed)
2. SSH password authentication was being rejected

8.2 Remove Stale Host Key

Command (Python, because Windows `ssh-keygen -R` syntax differs):

```
lines = open(
    r'C:\Users\valca\.ssh\known_hosts'
```

```
).readlines()
filtered = [l for l in lines
            if '192.168.1.77' not in l]
open(
    r'C:\Users\valca\.ssh\known_hosts', 'w'
).writelines(filtered)
print(f'Removed {len(lines)-len(filtered)} entries')
```

Output:

```
Removed 3 entries for 192.168.1.77
```

8.3 Second SSH Attempt (Still Failed)

Even after removing the stale host key, SSH password authentication was not accepted by the VCF Operations appliance. The `sshpass` utility was also not available on Windows.

8.4 Check VAMI

The user confirmed:

- VAMI at `https://192.168.1.77/admin` was accessible
- Logged in as **admin** with password `<your-password>`
- SSH was enabled in the VAMI settings
- There was only one toggle (no separate "Root SSH Login" toggle)

SSH from the Windows workstation was not possible. An alternative approach was needed.

8.5 Decision: Use vCenter Guest Operations API

Since we could not SSH into the appliance, we used the **vCenter Guest Operations API** instead. This API allows executing commands inside a guest VM through vCenter, as long as:

- VMware Tools is running in the guest
- You have vCenter administrator credentials
- You have the guest OS root credentials

9. Phase 6 — Import Certificate via vCenter Guest Operations API

9.1 How the Guest Operations API Works

The vCenter REST API provides an endpoint to start a process inside a guest VM:

```
POST https://<vcenter>/api/vcenter/vm/<vm-id>/
    guest/processes?action=create
```

The request body must include:

```
{
  "credentials": {
    "interactive_session": false,
    "type": "USERNAME_PASSWORD",
    "user_name": "root",
```

```

    "password": "*****"
  },
  "spec": {
    "path": "/bin/bash",
    "arguments": "-c \"<shell command>\""
  }
}

```

The API returns a **PID** (process ID) confirming the command was accepted.

9.2 Find the VCF Operations VM ID in vCenter

Before using Guest Operations, we needed the VM's internal ID.

Command (Python):

```

# Authenticate to vCenter
req = urllib.request.Request(
    'https://vcenter.lab.local/api/session',
    data=b'',
    headers={
        'Authorization': 'Basic ' +
        base64.b64encode(
            b'administrator@vsphere.local:'
            b'<your-password>'
        ).decode()
    },
    method='POST'
)
resp = urllib.request.urlopen(req, context=ctx)
session = json.loads(resp.read())

# List all VMs
req2 = urllib.request.Request(
    'https://vcenter.lab.local/api/vcenter/vm',
    headers={
        'vmware-api-session-id': session
    }
)
resp2 = urllib.request.urlopen(req2, context=ctx)
vms = json.loads(resp2.read())
for vm in vms:
    print(f'{vm["name"]} ({vm["vm"]})')

```

Output:

```

test (vm-1009)
vcenter (vm-18)
fleet (vm-4014)
vcf-ops (vm-4015)      <-- THIS ONE
collector (vm-4016)
nsx-manager (vm-58)
sddc-manager (vm-68)
logs (vm-69)
vCLS-fcd84d56-... (vm-8002)
vCLS-cdf14d56-... (vm-8003)

```

The VCF Operations VM is **vm-4015**.

9.3 Test Guest Operations with a Simple Command

Before running the certificate import, we verified that Guest Operations works by running a simple echo command.

Command (Python):

```
proc_data = json.dumps({
    "credentials": {
        "interactive_session": False,
        "type": "USERNAME_PASSWORD",
        "user_name": "root",
        "password": "<your-password>"
    },
    "spec": {
        "path": "/usr/bin/echo",
        "arguments": "hello"
    }
}).encode()

req = urllib.request.Request(
    'https://vcenter.lab.local/api/vcenter/vm/'
    'vm-4015/guest/processes?action=create',
    data=proc_data,
    headers={
        'vmware-api-session-id': session,
        'Content-Type': 'application/json'
    },
    method='POST'
)
resp = urllib.request.urlopen(req, context=ctx)
pid = json.loads(resp.read())
print(f'Process started, PID: {pid}')
```

Output:

```
Process started, PID: 643168
```

The PID was created. This confirms:

- vCenter can reach the VM
- VMware Tools is running in the guest
- The root credentials (<your-password>) are correct for the guest OS

9.4 Run the Certificate Import (6 Commands)

A Python script (`import_depot_cert.py`) was created to execute the following six commands inside the VCF Operations VM sequentially via the Guest Operations API.

Script location: `E:\VCF-Depot\import_depot_cert.py`

How to run it:

```
# bash on workstation (git-bash)
cd E:\VCF-Depot
py import_depot_cert.py
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Set-Location 'E:\VCF-Depot'
py .\import_depot_cert.py
```

Output:

```
vCenter session: OK

=== Step 1: Download depot server certificate ===
Started PID 645677

=== Step 2: Locate Java trust store ===
Started PID 645789

=== Step 3: Remove old alias if exists ===
Started PID 645901

=== Step 4: Import depot certificate into trust store ===
Started PID 646012

=== Step 5: Verify certificate was imported ===
Started PID 646123

=== Step 6: Restart vmware-vcops-analytics ===
Started PID 646234

=== Done. Wait 2-3 minutes, then retry. ===
```

Each PID confirms that the command was accepted and executed inside the guest VM. Below is what each command does.

Command 1 — Download the Depot Server's TLS Certificate

This command connects to the depot server, grabs its TLS certificate during the handshake, and saves it to a file.

Command executed inside the VM:

```
# Run on appliance only (executed inside vcf-ops VM via vCenter Guest Ops API)
echo | openssl s_client \
  -connect 192.168.1.160:8443 \
  2>/dev/null | openssl x509 > /tmp/depot.crt
```

What each part does:

Part	Purpose
echo \	Sends empty input to close the connection after the handshake
openssl s_client -connect 192.168.1.160:8443	Connects to the depot server via TLS and prints the server certificate
2>/dev/null	Suppresses OpenSSL diagnostic output
\ openssl x509	Filters the output to only the PEM-encoded certificate
> /tmp/depot.crt	Saves the certificate to a file on the appliance

Result: File `/tmp/depot.crt` created (1,062 bytes).

Command 2 — Verify Java Paths Exist

Command executed inside the VM:

```
# Run on appliance only (executed inside vcf-ops VM via vCenter Guest Ops API)
ls /usr/java/default/bin/keytool \
  /usr/java/default/lib/security/cacerts
```

What this checks: Confirms that the Java `keytool` command and the `cacerts` trust store file exist at the expected paths. VCF Operations uses OpenJDK 17.0.16 LTS.

Command 3 — Remove Old Certificate Alias (If It Exists)

Command executed inside the VM:

```
# Run on appliance only (executed inside vcf-ops VM via vCenter Guest Ops API)
/usr/java/default/bin/keytool \
  -delete \
  -alias depot-server \
  -keystore /usr/java/default/lib/security/cacerts \
  -storepass changeit \
  2>/dev/null; echo done
```

What each flag does:

Flag	Purpose
<code>-delete</code>	Deletes a certificate entry from the keystore
<code>-alias depot-server</code>	The name we assigned to our depot cert
<code>-keystore ../cacerts</code>	Path to the Java trust store
<code>-storepass changeit</code>	Default Java trust store password
<code>2>/dev/null</code>	Suppress "alias not found" error on first run
<code>; echo done</code>	Always exit successfully

Purpose: If this script is run a second time, the old entry is removed first to prevent "alias already exists" errors.

Command 4 — Import the Depot Certificate

Command executed inside the VM:

```
# Run on appliance only (executed inside vcf-ops VM via vCenter Guest Ops API)
/usr/java/default/bin/keytool \
  -importcert \
  -alias depot-server \
  -file /tmp/depot.crt \
  -keystore /usr/java/default/lib/security/cacerts \
  -storepass changeit \
  -noprompt
```

What each flag does:

Flag	Purpose
<code>-importcert</code>	Import a certificate into the keystore
<code>-alias depot-server</code>	Name for this certificate entry
<code>-file /tmp/depot.crt</code>	The certificate file from Command 1
<code>-keystore ../cacerts</code>	Path to the Java trust store
<code>-storepass changeit</code>	Trust store password

Flag	Purpose
-noprompt	Skip "Trust this certificate? [yes/no]" prompt

Result: Certificate added to the trust store.

Command 5 — Verify the Import

Command executed inside the VM:

```
# Run on appliance only (executed inside vcf-ops VM via vCenter Guest Ops API)
/usr/java/default/bin/keytool \
  -list \
  -alias depot-server \
  -keystore /usr/java/default/lib/security/cacerts \
  -storepass changeit
```

Expected output:

```
depot-server, Mar 23, 2026, trustedCertEntry,
Certificate fingerprint (SHA-256):
AA:A7:BB:F8:09:0C:D5:CF:18:30:33:D5:
8E:5F:CF:60:D6:F4:A7:55:2E:98:05:A9:
D4:94:1B:4C:AD:CA:2F:18
```

The fingerprint must match the depot server certificate's SHA-256 fingerprint.

Command 6 — Restart the Analytics Service

Command executed inside the VM:

```
# Run on appliance only (executed inside vcf-ops VM via vCenter Guest Ops API)
systemctl restart vmware-vcops-analytics
```

Why this is necessary: The Java Virtual Machine (JVM) loads the trust store into memory when it starts. After importing a new certificate into the `cacerts` file, the analytics service must be restarted so the JVM reloads the trust store. Without this restart, the running JVM continues to reject the depot certificate.

Recovery time: The analytics service takes approximately 2-3 minutes to fully restart and become responsive.

10. Phase 7 — Verify Certificate Import from Inside the VM

After the certificate import, we needed to verify that everything worked — from inside the VCF Operations VM. Since SSH was not available, a **diagnostic verification script** was used. This script runs commands inside the VM via Guest Operations API and retrieves the output by having the VM POST the results back to a temporary HTTP listener on the workstation.

10.1 How the Verification Works

Script location: `E:\VCF-Depot\diag2.py`

How to run it:


```
# bash on workstation (git-bash)
cd E:\VCF-Depot
py diag2.py
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Set-Location 'E:\VCF-Depot'
py .\diag2.py
```

How the callback mechanism works:

1. The script starts a temporary HTTP server on the workstation (e.g., port 9990)
2. It tells the VCF Operations VM to run a command and pipe the output to `curl -X POST` pointing back at port 9990 on the workstation
3. The workstation receives the output and prints it
4. The temporary HTTP server shuts down

Example of what runs inside the VM:

```
# Run on appliance only (executed inside vcf-ops VM via vCenter Guest Ops API)
curl -sk -o /dev/null -w '%{http_code}' \
https://192.168.1.160:8443/ | \
curl -sk -X POST -d @- \
http://192.168.1.160:9990/
```

This curls the depot, gets the HTTP status code, and POSTs that status code back to port 9990 on the workstation.

10.2 Verification Output

Command:

```
# bash on workstation (git-bash)
cd E:\VCF-Depot && py diag2.py
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Set-Location 'E:\VCF-Depot'
py .\diag2.py
```

Output:

1. Curl depot from VCF Ops:
200
2. Cert in Java trust store:
Warning: use -cacerts option to access
cacerts keystore
depot-server, Mar 23, 2026,
trustedCertEntry,
Certificate fingerprint (SHA-256):
AA:A7:BB:F8:09:0C:D5:CF:18:30:33:D5:
8E:5F:CF:60:D6:F4:A7:55:2E:98:05:A9:
D4:94:1B:4C:AD:CA:2F:18
3. Depot cert file on appliance:

```
-rw-r--r-- 1 root root 1062
Mar 23 23:27 /tmp/depot.crt
```

4. Java path:
openjdk version "17.0.16"
2025-07-15 LTS
5. Recent CASA log errors:
(empty – no errors)

What each test confirms:

Test	Result	Meaning
1	HTTP 200	VCF Ops VM can reach the depot server
2	Fingerprint matches	Certificate was successfully imported
3	File exists, 1062 bytes	Certificate was downloaded to the VM
4	OpenJDK 17.0.16	Java version confirmed
5	No errors	No CASA log entries about depot failures

Root Cause 2 (untrusted certificate) is now resolved.

11. Phase 8 — Third Failure: Depot URL Invalid

11.1 What the User Did

After the certificate import and service restart:

1. Went back to Fleet Management > Depot Configuration
2. Clicked CONFIGURE under Offline Depot
3. Selected Webserver
4. Entered **Repository URL:** `https://192.168.1.160:8443/Files/`
5. Entered **Username:** `admin`, **Password:** `admin`
6. Checked "I accept the imported certificate"
7. Clicked OK

11.2 What Happened

Error message:

```
Either the depot URL provided is partial
or invalid or not reachable or download
token is invalid. Check logs for more details.
```

This time we got an actual error message instead of a silent failure. The certificate fix is working (VCF Operations can now connect to the server), but something about the URL or content is wrong.

12. Phase 9 — Analyze Request Logs to Find the Real Problem

12.1 Check What the Server Received

The HTTPS server logs every request to `request_log.txt` in its serving directory.

Command:

```
# bash on workstation (git-bash) – reads Windows F:\ drive via /f/ mount
cat /f/request_log.txt | tail -10
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Get-Content -Path 'F:\request_log.txt' -Tail 10
```

Note: At this point the server was serving `F:\` (the user had previously asked to serve `F:\` because "files are on `F:\` drive").

Output:

```
192.168.1.78 - "GET /PROD/metadata/manifest
/v1/vcfManifest.json HTTP/1.1" 404 -

192.168.1.78 - "GET /Files/PROD/metadata
/manifest/v1/vcfManifest.json HTTP/1.1" 404 -

192.168.1.77 - "GET / HTTP/1.1" 200 -
192.168.1.77 - "GET / HTTP/1.1" 200 -
192.168.1.77 - "GET / HTTP/1.1" 200 -
```

12.2 What This Tells Us

Two critical discoveries:

Discovery 1: Fleet Management (192.168.1.78) is requesting `/PROD/metadata/manifest/v1/vcfManifest.json` and getting **404 Not Found**. This is because the server is serving `F:\`, and `F:\` does not have a `PROD\` directory.

Discovery 2: When the URL was set to `https://192.168.1.160:8443/Files/`, Fleet Management requested `/Files/PROD/metadata/manifest/v1/vcfManifest.json` — it appends `PROD/metadata/manifest/v1/vcfManifest.json` to whatever base URL you give it.

12.3 Where Is the PROD Directory?

Command:

```
# bash on workstation (git-bash) – checks Windows E:\VCF-Depot via /e/ mount
ls -la /e/VCF-Depot/PROD/metadata/manifest/\
v1/vcfManifest.json
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Get-Item 'E:\VCF-Depot\PROD\metadata\manifest\v1\vcfManifest.json' |
Format-List FullName, Length, LastWriteTime
```

Output:

```
-rw-r--r-- 1 valca 197610 136743
Oct  1 09:34
/e/VCF-Depot/PROD/metadata/manifest/
v1/vcfManifest.json
```

Command:

```
# bash on workstation
ls -la /f/PROD/
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Get-ChildItem -Path 'F:\PROD\' -Force -ErrorAction Stop
```

Output:

```
ls: cannot access '/f/PROD/': No such file
or directory
```

Command:

```
# bash on workstation
ls -la /f/Files/PROD/
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Get-ChildItem -Path 'F:\Files\PROD\' -Force -ErrorAction Stop
```

Output:

```
ls: cannot access '/f/Files/PROD/': No such
file or directory
```

Conclusion: The `PROD/` directory exists on `E:\VCF-Depot`, not on `F:\` or `F:\Files`. Two things need to change:

1. The server must serve `E:\VCF-Depot` (not `F:\`)
2. The URL must be `https://192.168.1.160:8443/` (not `/Files/`)

13. Phase 10 — Correct the Serving Directory and URL

13.1 Change the Serving Directory in the Script

File: `E:\VCF-Depot\https_server.py`

Before (line 83):

```
os.chdir('F:/')
```

After (line 83):

```
os.chdir('E:/VCF-Depot')
```

13.2 Kill the Old Server

Command:

```
# bash on workstation (git-bash)
taskkill /F /PID $(netstat -ano | \
  grep ':8443.*LISTENING' | awk '{print $5}')
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
$pid8443 = (Get-NetTCPConnection -LocalPort 8443 -State Listen -ErrorAction
  SilentlyContinue).OwningProcess
if ($pid8443) { Stop-Process -Id $pid8443 -Force }
```

Output:

```
SUCCESS: The process has been terminated.
```

Verify port is free:

```
# bash on workstation
netstat -an | grep 8443
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Get-NetTCPConnection -LocalPort 8443 -ErrorAction SilentlyContinue |
  Select-Object LocalAddress, LocalPort, State, OwningProcess
```

Output: (empty — port is free)

13.3 Start the Server with the Correct Directory

Command:

```
# bash on workstation
cd /e/VCF-Depot && py https_server.py
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Set-Location 'E:\VCF-Depot'
py .\https_server.py
```

13.4 Verify the Manifest Is Now Accessible

Command:

```
# bash on workstation
curl -sk -o /dev/null -w "%{http_code}" \
  https://192.168.1.160:8443/PROD/metadata/\
  manifest/v1/vcfManifest.json
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true } # PS 5.1 self-signed
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
(Invoke-WebRequest -Uri 'https://192.168.1.160:8443/PROD/metadata/manifest/v1/vcfManifest.json' -
UseBasicParsing).StatusCode
# OR: curl.exe -sk -o NUL -w "%{http_code}"
https://192.168.1.160:8443/PROD/metadata/manifest/v1/vcfManifest.json
```

Output:

```
200
```

The manifest file now returns HTTP 200.

13.5 Configure the Depot in Fleet Management

1. Open VCF Operations in a browser
2. Navigate to **Fleet Management > Depot Configuration**
3. Click **CONFIGURE** under Offline Depot
4. Fill in the dialog:

Field	Value
OfflineDepotType	Webserver
Repository URL	https://192.168.1.160:8443/
Username	admin
Password	admin

5. Check **"I accept the imported certificate"**
6. Click **VIEW CERTIFICATE DETAILS** to confirm the certificate fingerprint matches
7. Click **OK**

13.6 Result

The offline depot now shows as **Active**.

14. Phase 11 — Final Validation

14.1 Verify from the Workstation

Test 1 — Server is listening:

```
# bash on workstation
netstat -an | grep 8443
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Get-NetTCPConnection -LocalPort 8443 -ErrorAction SilentlyContinue |
    Select-Object LocalAddress, LocalPort, State, OwningProcess
```

```
TCP      0.0.0.0:8443      0.0.0.0:0        LISTENING
```

Test 2 — Root URL returns directory listing:

```
# bash on workstation
curl -sk -o /dev/null -w "%{http_code}" \
https://192.168.1.160:8443/
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true } # PS 5.1 self-signed
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
(Invoke-WebRequest -Uri 'https://192.168.1.160:8443/' -UseBasicParsing).StatusCode
# OR: curl.exe -sk -o NUL -w "%{http_code}" https://192.168.1.160:8443/
```

```
200
```

Test 3 — Manifest file is accessible:

```
# bash on workstation
curl -sk -o /dev/null -w "%{http_code}" \
https://192.168.1.160:8443/PROD/metadata/\
manifest/v1/vcfManifest.json
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation (cert-bypass + TLS already set in Test 2)
(Invoke-WebRequest -Uri 'https://192.168.1.160:8443/PROD/metadata/manifest/v1/vcfManifest.json' -
UseBasicParsing).StatusCode
# OR: curl.exe -sk -o NUL -w "%{http_code}"
https://192.168.1.160:8443/PROD/metadata/manifest/v1/vcfManifest.json
```

```
200
```

Test 4 — No authentication required:

```
# bash on workstation
curl -sk -o /dev/null -w "%{http_code}" \
https://192.168.1.160:8443/
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation (cert-bypass + TLS already set above)
(Invoke-WebRequest -Uri 'https://192.168.1.160:8443/' -UseBasicParsing).StatusCode
# OR: curl.exe -sk -o NUL -w "%{http_code}" https://192.168.1.160:8443/
```

200

(No `-u admin:admin` flag — still returns 200)

14.2 Verify from Inside VCF Operations VM

Command:

```
# bash on workstation (git-bash) – script invokes Guest Ops API into vcf-ops VM
cd E:\VCF-Depot && py diag2.py
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Set-Location 'E:\VCF-Depot'
py .\diag2.py
```

Output:

1. Curl depot from VCF Ops:
200
2. Cert in Java trust store:
depot-server, Mar 23, 2026,
trustedCertEntry,
Certificate fingerprint (SHA-256):
AA:A7:BB:F8:09:0C:D5:CF:18:30:33:D5:
8E:5F:CF:60:D6:F4:A7:55:2E:98:05:A9:
D4:94:1B:4C:AD:CA:2F:18
3. Depot cert file on appliance:
-rw-r--r-- 1 root root 1062
Mar 23 23:27 /tmp/depot.crt
4. Java path:
openjdk version "17.0.16"
2025-07-15 LTS
5. Recent CASA log errors:
(empty – no errors)

14.3 Verify Request Logs Show 200s

Command:

```
# bash on workstation (git-bash) – reads Windows E:\VCF-Depot via /e/ mount
cat /e/VCF-Depot/request_log.txt | tail -5
```

PowerShell equivalent:


```
# PowerShell 5.1+ on Windows workstation
Get-Content -Path 'E:\VCF-Depot\request_log.txt' -Tail 5
```

Output:

```
192.168.1.78 - "GET /PROD/metadata/manifest
/v1/vcfManifest.json HTTP/1.1" 200 -
192.168.1.78 - "GET /PROD/metadata/
productVersionCatalog/v1/
productVersionCatalog.json HTTP/1.1" 200 -
```

Fleet Management (192.168.1.78) is now successfully polling the depot and receiving HTTP 200 for both the manifest and the product version catalog.

15. Summary of All Four Root Causes

#	Root Cause	Symptom	Fix
1	HTTPS server required Basic Auth	HTTP 401 error in UI	Removed <code>authenticate()</code> calls from <code>do_HEAD()</code> , <code>do_GET()</code> , <code>do_POST()</code>
2	Self-signed cert not in Java trust store	Dialog closed silently, no save, no error	Imported cert via <code>keytool -importcert</code> using vCenter Guest Operations API
3	Server was serving <code>F:\</code> instead of <code>E:\VCF-Depot</code>	Manifest file returned 404	Changed <code>os.chdir('F:/')</code> to <code>os.chdir('E:/VCF-Depot')</code>
4	URL had <code>/Files/</code> suffix	Fleet appended <code>PROD/...</code> to <code>/Files/</code> , got 404	Changed URL to <code>https://192.168.1.160:8443/</code> (no suffix)

Order of resolution matters. Root Cause 1 had to be fixed first (otherwise you cannot test anything). Root Cause 2 had to be fixed second (otherwise the UI fails silently). Root Causes 3 and 4 were discovered together through request log analysis.

16. Lessons Learned

16.1 Fleet Management Does Not Always Show Error Messages

When Fleet Management cannot validate a depot URL due to a TLS certificate trust failure, the UI dialog closes without saving and without displaying any error. The only way to diagnose this is to check the depot server's request log or the appliance logs.

Recommendation: Always have the depot server's request log open in a separate terminal window when configuring the depot:

```
# bash on workstation (git-bash)
tail -f E:\VCF-Depot\request_log.txt
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Get-Content -Path 'E:\VCF-Depot\request_log.txt' -Wait -Tail 0
```

16.2 Fleet Management Expects a Specific Directory Structure

The offline depot URL must serve the VCF metadata structure. Fleet Management validates the depot by requesting:

```
<url>/PROD/metadata/manifest/v1/vcfManifest.json
```

If this file does not exist at the URL root, configuration fails. This file comes from the `vcf-*-offline-depot-metadata.zip` downloaded from Broadcom Support.

16.3 Self-Signed Certificates Must Be Imported into the Java Trust Store

VCF Operations runs on OpenJDK. Any HTTPS endpoint it connects to must have a certificate that is either:

- Signed by a public CA (trusted by default in Java's `cacerts`)
- Manually imported into `/usr/java/default/lib/security/cacerts`

After importing, the `vmware-vcops-analytics` service must be restarted.

16.4 vCenter Guest Operations API Is an Alternative to SSH

When SSH is unavailable, the vCenter Guest Operations API can execute commands inside any VM that has VMware Tools running. The API endpoint is:

```
POST https://<vcenter>/api/vcenter/vm/
<vm-id>/guest/processes?action=create
```

Requirements:

- vCenter administrator credentials
- Guest OS credentials (root)
- VMware Tools running in the guest

16.5 The Depot URL Must Not Include a Subdirectory

Fleet Management appends `PROD/metadata/manifest/v1/vcfManifest.json` to the base URL. If you set the URL to `https://192.168.1.160:8443/Files/`, the resulting request becomes `/Files/PROD/metadata/...` which is wrong. The URL must point to the parent directory of `PROD/`.

16.6 The Fleet Management UI Requires Username and Password

Even when the web server does not enforce authentication, the Fleet Management Depot Configuration dialog requires a username and password. These fields cannot be left blank. Enter any value (e.g., `admin / admin`) — the server will ignore them.

17. Appendix A — Final HTTPS Server Script

File: `E:\VCF-Depot\https_server.py`

```

#!/usr/bin/env python3
"""
HTTPS server for VCF Offline Depot
Serves files over TLS 1.2+ for Fleet Management
"""

import http.server
import ssl
import os
import base64
import socketserver
from functools import partial

# Configuration
PORT = 8443
CERT_FILE = 'E:/VCF-Depot/server.crt'
KEY_FILE = 'E:/VCF-Depot/server.key'
USERNAME = 'admin'
PASSWORD = 'admin'

class AuthHandler(
    http.server.SimpleHTTPRequestHandler
):
    protocol_version = "HTTP/1.1"

    def __init__(self, *args,
                 directory=None, **kwargs):
        super().__init__(
            *args, directory=directory, **kwargs
        )

    def do_HEAD(self):
        super().do_HEAD()

    def do_GET(self):
        super().do_GET()

    def do_POST(self):
        content_length = int(
            self.headers.get('Content-Length', 0)
        )
        self.rfile.read(content_length)
        self.send_response(200)
        self.send_header(
            'Content-Type', 'application/json'
        )
        self.send_header('Connection', 'close')
        self.end_headers()
        self.wfile.write(b '{"status": "ok"}')

    def log_message(self, format, *args):
        msg = (f"{self.client_address[0]}"
              f" - {format % args}")
        print(msg)
        with open('request_log.txt', 'a') as f:
            f.write(msg + '\n')

class ThreadedHTTPServer(
    socketserver.ThreadingMixIn,
    http.server.HTTPServer
):
    daemon_threads = True

def run_server():
    os.chdir('E:/VCF-Depot')

    context = ssl.SSLContext(
        ssl.PROTOCOL_TLS_SERVER
    )
    context.minimum_version = (
        ssl.TLSVersion.TLSv1_2

```

```

    )
    context.maximum_version = (
        ssl.TLSVersion.TLSv1_3
    )
    context.load_cert_chain(CERT_FILE, KEY_FILE)

    handler = partial(
        AuthHandler, directory=os.getcwd()
    )
    server = ThreadedHTTPServer(
        ('0.0.0.0', PORT), handler
    )
    server.socket = context.wrap_socket(
        server.socket, server_side=True
    )

    print("VCF Offline Depot Server")
    print("=====")
    print(f"Serving: {os.getcwd()}")
    print(f"URL: https://192.168.1.160:{PORT}/")
    print(f"TLS: 1.2 - 1.3")
    print("Press Ctrl+C to stop")

    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nStopped.")
        server.shutdown()

if __name__ == '__main__':
    run_server()

```

To start the server:

```

# bash on workstation (git-bash)
cd E:\VCF-Depot
py https_server.py

```

PowerShell equivalent:

```

# PowerShell 5.1+ on Windows workstation
Set-Location 'E:\VCF-Depot'
py .\https_server.py

```

To stop the server: Press `Ctrl+C` in the terminal.

18. Appendix B — Certificate Import Script

File: E:\VCF-Depot\import_depot_cert.py

This script imports the depot server's TLS certificate into the VCF Operations Java trust store using the vCenter Guest Operations API. Run this script any time the depot certificate is regenerated.

Prerequisites:

- Python 3.x on the Windows workstation
- vCenter must be running and accessible
- VMware Tools must be running on the vcf-ops VM
- The depot server must be running on port 8443

To run:

```
# bash on workstation (git-bash)
cd E:\VCF-Depot
py import_depot_cert.py
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Set-Location 'E:\VCF-Depot'
py .\import_depot_cert.py
```

What it does (6 steps):

1. Downloads the depot cert from 192.168.1.160:8443 to `/tmp/depot.crt` on the appliance
2. Verifies the Java keytool and cacerts paths exist
3. Removes any existing `depot-server` alias (safe to re-run)
4. Imports the certificate with alias `depot-server`
5. Verifies the import by listing the alias
6. Restarts `vmware-vcops-analytics`

Full script source: See `E:\VCF-Depot\import_depot_cert.py`

Variables to change for a different environment:

Variable	Current Value	Description
VCENTER	vcenter.lab.local	vCenter FQDN or IP
VM_ID	vm-4015	VM ID of VCF Operations
VC_USER	administrator@vsphere.local	vCenter admin user
VC_PASS	*****	vCenter admin password
GUEST_USER	root	VCF Ops VM root user
GUEST_PASS	*****	VCF Ops VM root password

19. Appendix C — Diagnostic Verification Script

File: `E:\VCF-Depot\diag2.py`

This script runs 5 diagnostic tests inside the VCF Operations VM and retrieves the results back to the workstation using a callback mechanism.

To run:

```
# bash on workstation (git-bash)
cd E:\VCF-Depot
py diag2.py
```

PowerShell equivalent:

```
# PowerShell 5.1+ on Windows workstation
Set-Location 'E:\VCF-Depot'
py .\diag2.py
```

What it tests:

Test	What It Checks
1	Can VCF Ops VM curl the depot? (expects HTTP 200)
2	Is the depot cert in the Java trust store? (expects fingerprint)
3	Does <code>/tmp/depot.crt</code> exist on the appliance?
4	What Java version is installed?
5	Are there any recent CASA log errors?

How the callback works:

1. The script starts a temporary HTTP listener on the workstation (ports 9990-9994)
2. It runs a command inside the VM via Guest Operations API
3. The command pipes its output to `curl -X POST` pointing back at the workstation
4. The workstation receives and prints the output
5. The temporary listener shuts down

Full script source: See `E:\VCF-Depot\diag2.py`

20. Document Information

Field	Value
Document Title	Root Cause Analysis — VCF Offline Depot Configuration
Prepared By	Virtual Control LLC
Original Date	March 23, 2026
Document Version	1.1
Last Updated	May 7, 2026
Classification	Internal — Lab Environment

Field	Value
VCF Version	9.0.1
Revision History	v1.0 (March 23, 2026) — Initial release covering the four root causes (HTTP 401, untrusted self-signed cert, wrong serving directory, URL with /Files/ suffix) and their resolution via vCenter Guest Operations API certificate import. v1.1 (May 7, 2026) — Option B PowerShell-sibling pass: added top-of-doc callout linking to <code>../POWERSHELL-TRANSLATION-REFERENCE.md</code> ; added paste-ready PowerShell sibling blocks beneath every workstation-targeted bash block (<code>netstat</code> , <code>taskkill</code> , <code>curl</code> , <code>cat</code> , <code>tail</code> , <code>ls</code> , <code>cd</code> & <code>py</code> , <code>ssh</code>); flagged appliance-only blocks (<code>openssl s_client</code> , <code>keytool</code> , <code>systemctl</code> , <code>callback-curl</code>) with <code># Run on appliance only</code> context comment and no PS sibling.

This report documents every command executed, every output received, and every configuration change made on March 23, 2026 to configure the VCF Operations Fleet Management offline depot. All steps can be replicated using the commands and scripts documented above.

Prepared by Virtual Control LLC